Final Project Report: Big Data Analytics & MLOps for Asset Pricing

Team: *Tam Dang, Ayush Kumar, Agniva Mondal, Shiv Raj Bangad*

Executive Summary

This project leverages Apache Spark and Databricks to construct an "All-Weather" institutional trading strategy by processing large-scale financial data and extracting predictive insights. The core objective is to forecast monthly stock excess returns while mitigating downside risk during unprecedented market volatility. By utilizing PySpark for distributed data ingestion, Spark MLlib for predictive modeling, and MLflow for strict experiment tracking, the project adheres to end-to-end MLOps best practices . The primary business outcome is a "Macro-Turbulence Gate" that dynamically routes capital between complex Gradient Boosted Trees and high-bias Linear models, ensuring robust capital preservation during financial crises.

Phase 1: Data Selection & Problem Definition

- **Business Problem:** Standard predictive asset pricing models often fail during periods of unprecedented market turbulence, such as the 2020 COVID-19 crash. The objective of this project is to implement a regression model that accurately forecasts monthly stock excess returns. By dynamically allocating capital based on real-time macroeconomic turbulence, the strategy mitigates downside risk while capturing bull-market upside.
- **Dataset Overview:** The project utilizes an empirical finance asset pricing dataset containing structured monthly equity data for U.S. publicly traded firms.
- **Target Variable:** The primary target variable for the predictive models is ret_excess, representing the monthly excess return of the stock.
- **Feature Space:** The dataset incorporates 94 distinct firm-specific characteristics (e.g., momentum, book-to-market) that have been rank-transformed to handle outliers and ensure stationarity. It also includes 8 macroeconomic predictors (e.g., dividend-price ratio, treasury bill rate) that capture the aggregate state of the economy.
- **Categorical Controls:** To control for industry-specific fixed effects, the data includes one-hot encoded variables based on the first two digits of Standard Industrial Classification (SIC) codes.

Phase 2: Big Data Ingestion & Preprocessing

- **Custom PyArrow Schema Mapping:** The raw data, stored in partitioned Parquet format, contained nanosecond timestamps that are incompatible with default Spark vectorization. To bypass schema inference crashes securely, PyArrow was utilized to

read the native schema and map it to a custom Spark StructType, casting the problematic timestamps to standard Long integers prior to ingestion.

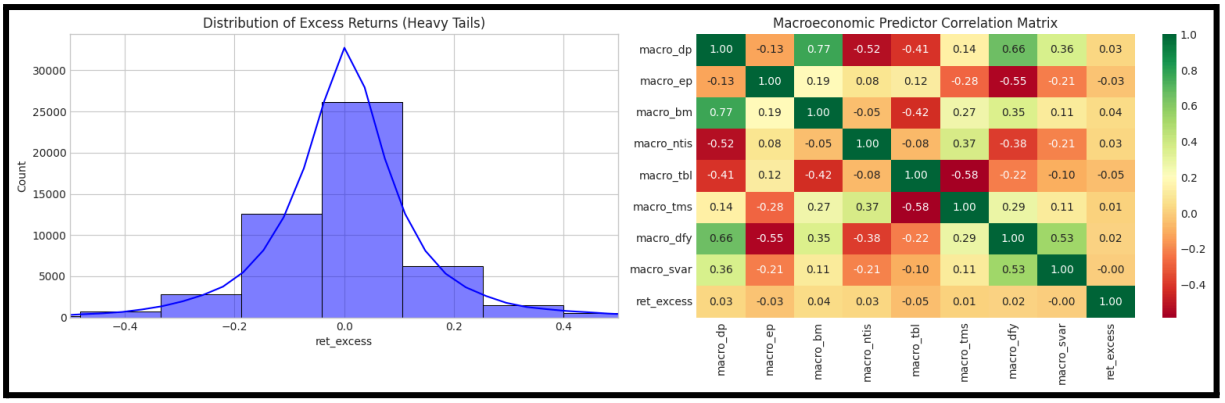
- **Distributed Spark SQL Filtering:** Following successful ingestion into Databricks via PySpark, Spark SQL was leveraged to perform data exploration and cleaning. The dataset was strictly filtered to the 2000–2021 evaluation window, and all records containing null values in the target variable (ret_excess) were removed to ensure data integrity.
- **Timestamp Transformation & Optimization:** The temporary integer timestamps were cast back into standard Spark TimestampType objects using Spark DataFrame functions (F.to_timestamp). Finally, to optimize data storage and accelerate downstream retrieval, the cleaned and consolidated DataFrame was written back to the Databricks File System (DBFS) in an optimized Parquet format.

Phase 3: Exploratory Data Analysis (EDA)

- **Distributed Summary Statistics:** Distributed summary statistics were generated across the entire dataset using Spark queries (.summary()) to analyze the central tendencies, variances, and spread of the excess returns and macroeconomic indicators.

```
Target Variable (ret_excess) Summary Statistics:
+-------+--------------------+--------------------+--------------------+
|summary|          ret_excess|           macro_dp|           macro_tbl|
+-------+--------------------+--------------------+--------------------+
|  count|             1027681|             1027681|             1027681|
|   mean|0.009434446646381552| -3.988849148366859|0.014562447782922568|
| stddev| 0.18519051353777954|0.17615050216508857|0.015611610200738748|
|    min| -0.9971329999999999| -4.436386559722948|               1.0E-4|
|    25%|           -0.064877| -4.066850129556806|               0.001|
|    50%|            0.002595|-3.9761522963969727|              0.0101|
|    75%| 0.06931899999999999| -3.899372592549001|              0.0219|
|    max|  19.881490000000003|  -3.28100756237336|              0.0515|
+-------+--------------------+--------------------+--------------------+
```

- **Target Distribution Analysis:** To analyze distributions safely without exhausting cluster memory, a representative 5% sample was extracted into a Pandas DataFrame. Analysis of the target variable (ret_excess) revealed significant heavy tails, a common characteristic in financial datasets that confirms the necessity for robust, non-linear predictive models capable of handling outliers.
- **Predictor Collinearity:** A correlation matrix was computed on the macroeconomic variables. The analysis identified severe multi-collinearity among several macro indicators. This insight directly informed the modeling strategy, justifying the implementation of tree-based ensembles and regularized linear models, both of which are resistant to correlated feature spaces.



Phase 4: Predictive Modeling using Spark MLlib

- **Modeling Approach & Feature Assembly:** A regression framework was implemented to forecast the continuous ret_excess target. To prepare the data for MLlib, a VectorAssembler was used to concatenate the 94 firm characteristics, 8 macroeconomic indicators, and industry SIC codes into a single unified feature vector.
- **Algorithm Selection:** Spark MLlib was leveraged to construct a GBTRegressor (Gradient Boosted Trees). Tree-based ensembles are highly effective at capturing non-linear interactions across the complex "factor zoo." Additionally, a LinearRegression model was trained to serve as a high-bias, safe baseline for the gating strategy.
- **Hyperparameter Tuning & Cross-Validation:** To prevent overfitting and optimize predictive accuracy, hyperparameter tuning was conducted using Spark's CrossValidator and ParamGridBuilder. The grid search iteratively tested varying maximum tree depths (3 and 5) and iteration counts across multiple folds. Model performance was strictly evaluated using the Root Mean Squared Error (RMSE) metric.
- **Chronological Data Splitting:** Due to the time-series nature of financial data, a strict chronological split was enforced (training on data up to 2018, evaluating out-of-sample on data post-2018) to prevent look-ahead bias and data leakage.

Phase 5: MLOps Best Practices

- **Pipeline Automation:** To ensure end-to-end reproducibility, the data processing steps (VectorAssembler) and the predictive algorithms were sequentially linked using Spark Pipeline objects. This automation guarantees that incoming test data receives the exact same transformations as the training data.
- **Experiment Tracking with MLflow:** Databricks MLflow was integrated to rigorously monitor the model training lifecycle. All model configurations, hyperparameter combinations, and out-of-sample RMSE evaluation metrics were logged automatically to the MLflow experiment registry, allowing for comprehensive run comparison and version control.
- **Cluster Environment Management:** To comply with Databricks Unity Catalog security protocols on shared clusters, the SPARKML_TEMP_DFS_PATH

environment variable was explicitly routed to an authorized volume path, allowing MLlib to safely cache intermediate cross-validation data.

Phase 6: Insights & Business Recommendations

- **Key Findings:** The evaluation demonstrated that the GBTRegressor successfully captures complex, non-linear financial patterns and outperforms baseline linear models during normal, low-volatility market regimes.
- **Challenges Identified:** Complex machine learning algorithms, particularly deep trees, are highly sensitive to regime shifts. They are prone to overfitting historical relationships and can fail catastrophically during unprecedented macroeconomic shocks, such as the market crash of early 2020.
- **Actionable Business Recommendation (Regime Gating):** To solve this challenge, we engineered an institutional "Macro-Turbulence Gate". By calculating the real-time Mahalanobis Distance of current macroeconomic variables against their historical covariance matrix, the system establishes a quantitative index of market stress.
- **Strategic Outcome:** A Sigmoid Transition Gate was applied to this index. When macro-turbulence spikes beyond the 85th percentile, the algorithm seamlessly deleverages portfolio exposure from the GBTRegressor and reallocates capital to the LinearRegression baseline. This provides actionable business value by ensuring strict capital preservation during black-swan events while maximizing returns during bull markets.

References

- [1] Gu, S., Kelly, B., & Xiu, D. (2020). Empirical asset pricing via machine learning. *The Review of Financial Studies*, 33(5), 2223-2273.
- [2] Green, J., Hand, J. R., & Zhang, X. F. (2017). The characteristics that provide independent information about average U.S. monthly stock returns. *The Review of Financial Studies*, 30(12), 4389-4436.
- [3] Welch, I., & Goyal, A. (2008). A comprehensive look at the empirical performance of equity premium prediction. *The Review of Financial Studies*, 21(4), 1455-1508.
- [4] Ye, H. J., Liu, S. Y., Cai, H. R., Zhou, Q. L., & Zhan, D. C. (2024). A Closer Look at Deep Learning Methods on Tabular Datasets. *arXiv preprint arXiv:2407.00956*.
- [5] Liu, Y., Luo, Y., Wang, Z., & Zhang, X. (2026). Uncertainty-Adjusted Sorting for Asset Pricing with Machine Learning. *SSRN Working Paper No. 6013574*.